



Project-04

Managing Web Application Releases Across Different Environments Using GitHub Actions

Prepared by:

REPALLE.VINAYKUMAR

Mail Id:vinaykumarrepalle93gmail.com

Batch: 149

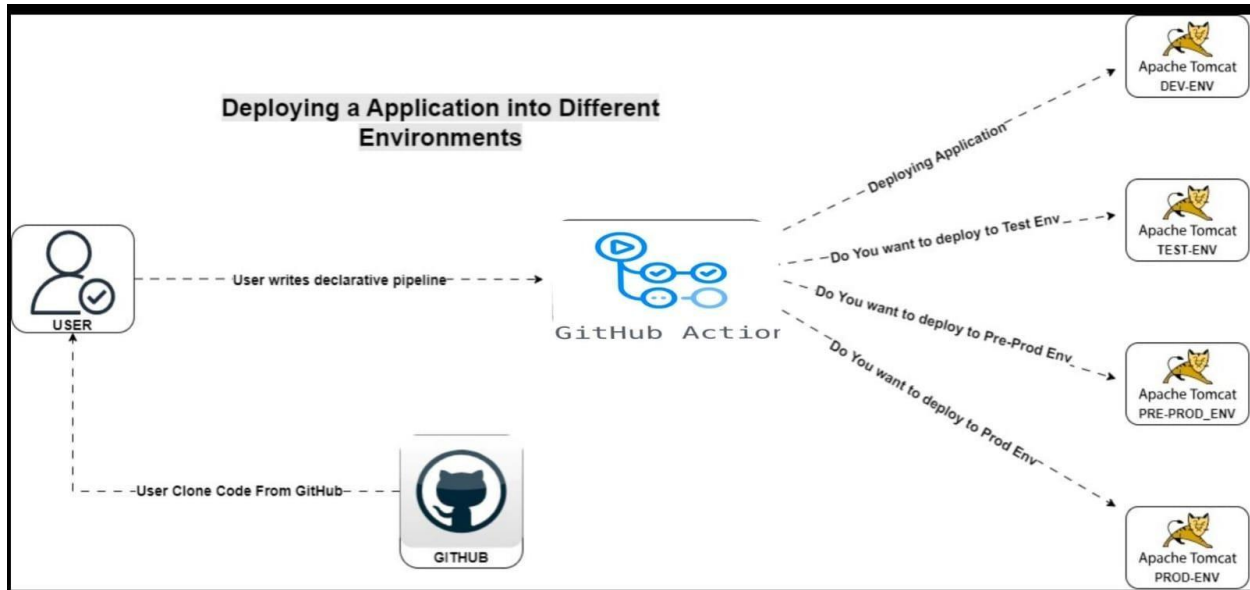


Fig: 1 Hosting Web Application in Apache Tomcat

Introduction to the Project:

This project illustrates the automation of web application deployment across multiple environments—namely development, testing, pre-production, and production—by leveraging **GitHub Actions**. The primary objective is to establish a consistent and dependable CI/CD pipeline seamlessly integrated with **Apache Tomcat servers**.

Project Objectives:

- Implement automated deployment pipelines through GitHub Actions.
- Facilitate deployment across multiple environments (Development, Testing, Pre-Production, Production) driven by user configuration.
- Reduce human intervention errors while improving the speed and efficiency of deployment cycles.

Provisioning AWS EC2 for Deployment

The project uses four Amazon EC2 machines, each running Apache Tomcat, to handle different environments.

- **DEV** – Development server
- **TEST** – Testing server
- **PRE-PROD** – Pre-Production server
- **PROD** – Production server

Apache Tomcat was installed on the development server, and an AMI was built from it to quickly set up the test, pre-production, and production servers.

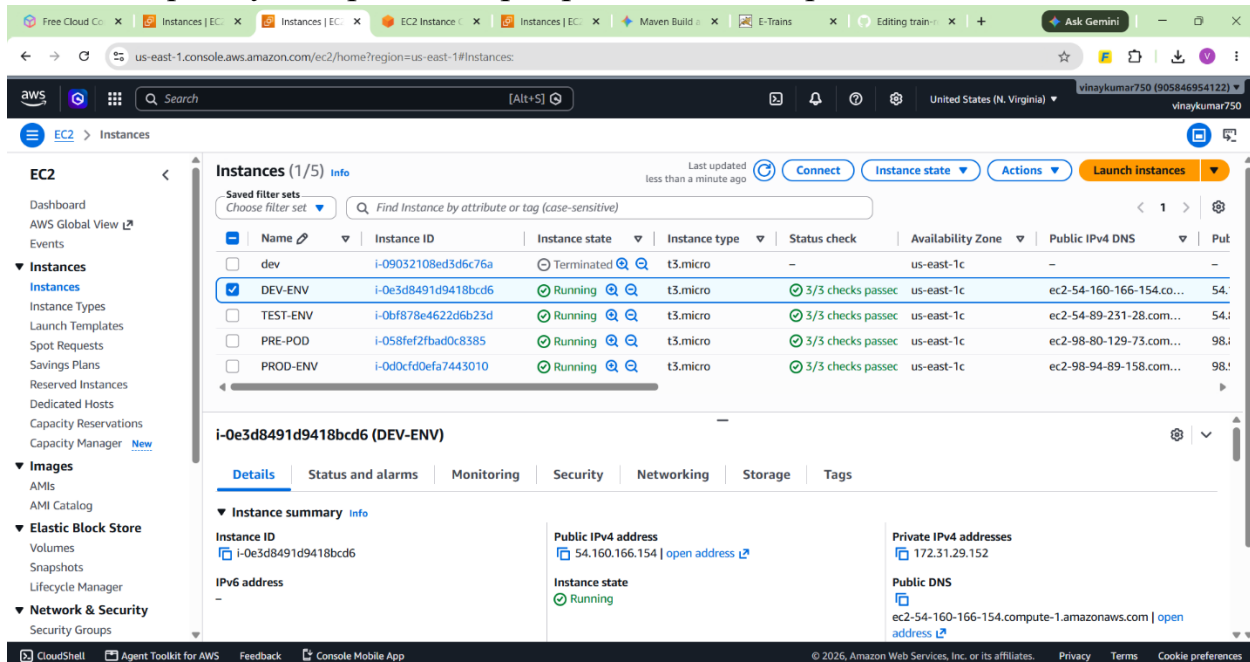
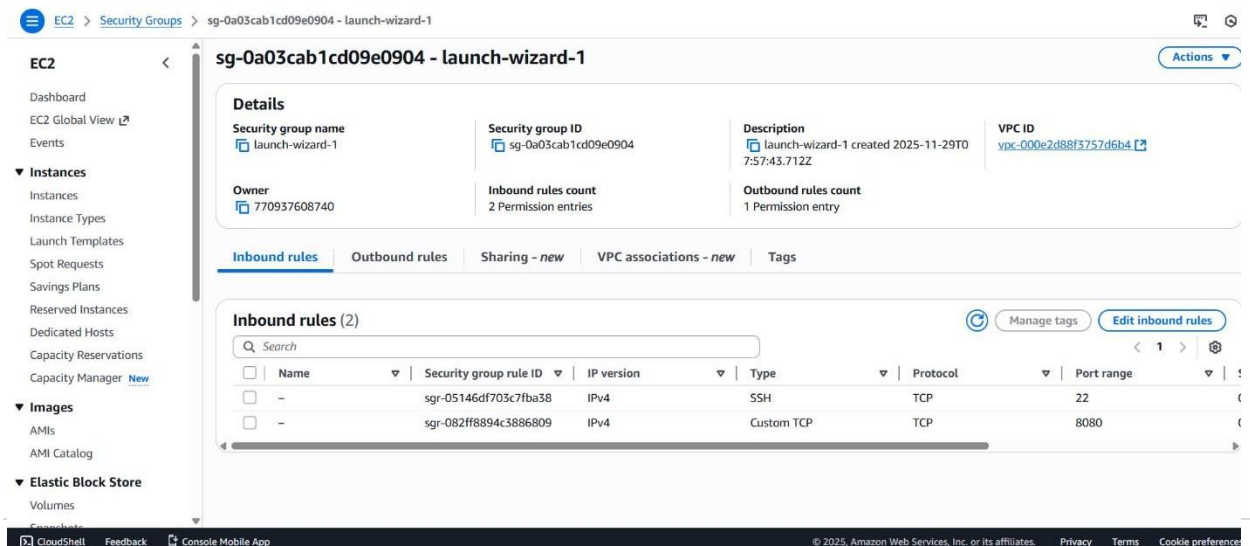


Fig: 2 Dev-Tomcat Instance

- Go to the AWS EC2 Console and click **Launch Instance**.
- Name the server **Dev-Tomcat** and pick **Ubuntu** as the AMI.
- Use **t2.micro** for demo setups. For real use, choose **t3.small** or **t3.medium**. Create or pick a key pair and save it safely.
- In the network settings, use the default or custom VPC, pick a subnet, and turn on auto public IP. Allow **Port 22** for SSH and **Port 8080** for Tomcat in the security group.



Installation of Java & Apache Tomcat

To configure the Dev-Tomcat instance, connect to it through Git Bash and first become the root user by typing **sudo -i**

Then update the server packages with **apt update -y** and install Java using **apt install openjdk-11-jdk -y**.

You can confirm the installation by running **java -version**.

Next, download Apache Tomcat 9 with **wget**

<https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.112/bin/apache-tomcat-9.0.112.tar.gz>

Check if the file has been downloaded using **ls**.

Extract the archive by executing **tar -xf apache-tomcat-9.0.112.tar.gz** then rename the folder with **mv apache-tomcat-9.0.112 tomcat**.

Move into the Tomcat directory using **cd tomcat/** and then into the bin folder with **cd bin/**.

Start the Tomcat server by running **./startup.sh** and stop it when required using **./shutdown.sh**.

Before updating the tomcat-users.xml file, comment out or edit the access restriction lines in the **manager** and **host-manager** configuration files. Inside the conf directory, add the following roles and users:

```
<role rolename="manager-gui"/>
<user username="tomcat" password="s3cret" roles="manager-gui"/>
<role rolename="manager-script"/>
<user username="deployer" password="deployer" roles="manager-script"/>
```

Here, the rolename defines the type of access (like manager-gui or manager-script), while the user element specifies the username, password, and the roles assigned to that user.

You can customize these values to match your own security requirements, ensuring that each user has the correct level of access to Tomcat's management tools.

You can start the Tomcat server from the bin directory by running **./startup.sh**. Once the server is running, copy the public IP address of the Dev-Tomcat instance and paste it into a browser to confirm that Tomcat has been successfully installed on the Development server.

After completing the setup of the DEV instance, clone the application code from GitHub inside the server by executing **git clone <repository-url>**.

Once the DEV server is fully configured, you need to create an Amazon Machine Image (AMI) from the instance so that it can be reused to provision the other environments.

To do this, go to the EC2 console, select the DEV instance, choose **Actions** → **Image and templates** → **Create Image (AMI)**, and provide a name such as *tomcat-base-image*.

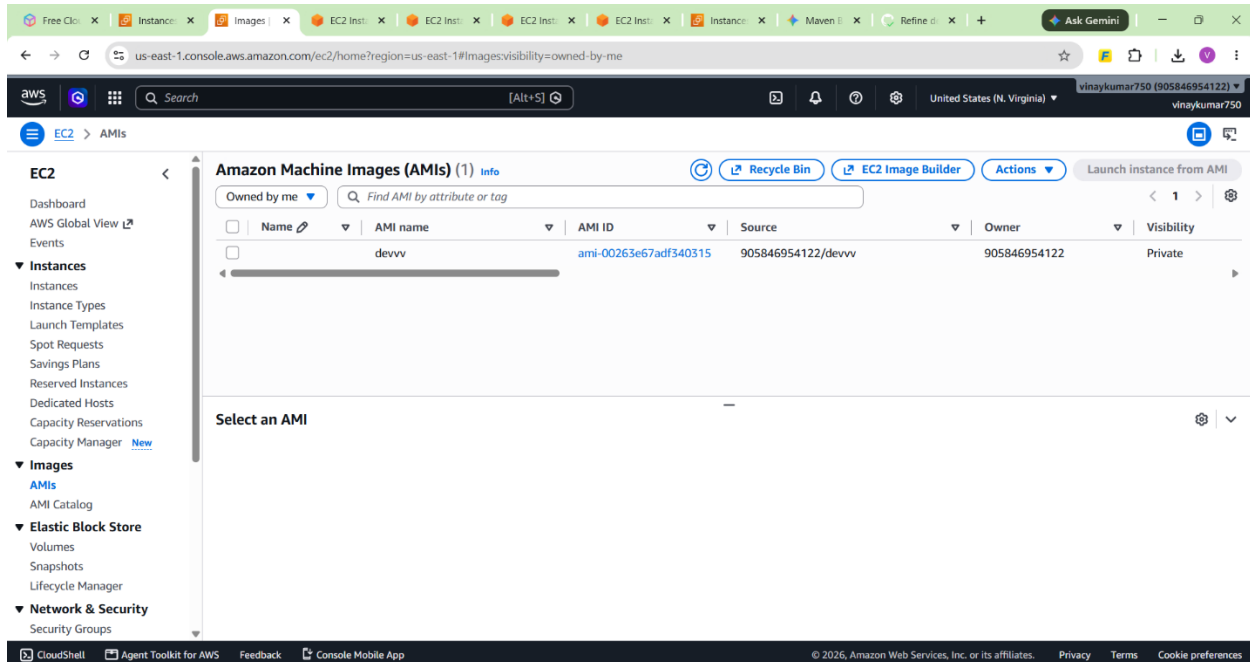


Fig: 3 Amazon Machine Image

After the AMI is created, you can launch three new EC2 instances from it, which will serve as the TEST, PRE-PROD, and PROD servers.

By using the same Amazon Machine Image, each of these servers inherits the identical Java installation, Apache Tomcat configuration, user roles, and folder structure from the DEV instance, ensuring consistency across all stages of deployment and eliminating the need for repetitive setup or reconfiguration.

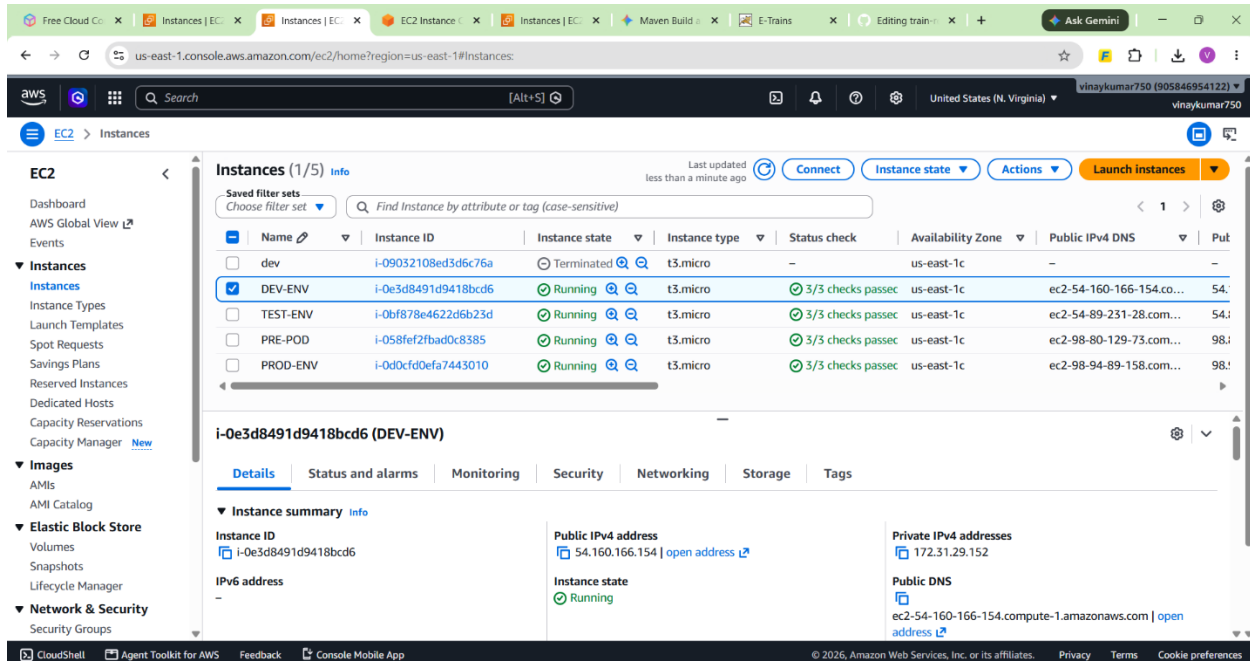


Fig: 3(i) Launched Instances of Test, Pre-Prod, and Prod

CI/CD Pipeline Setup with GitHub Actions

In GitHub Actions, four deployment environments were defined—DEV, TEST, PRE-PROD, and PROD—and each of these environments was mapped to its corresponding Apache Tomcat server hosted on AWS EC2 by navigating to *Repository* → *Settings* → *Environments* → *New Environment*.

For the deployment workflow, the DEV, TEST, and PRE-PROD environments were configured to run automatically without requiring any manual approval, while the PROD environment was protected with a rule that enforces at least one manual approval before deployment can proceed.

Environment Credentials and Variables in GitHub

In GitHub Actions, environment secrets and variables are created to securely store the connection details required for each Apache Tomcat server running on AWS EC2 instances.

These secrets are defined within the repository settings under *Environments*, ensuring that sensitive information such as server credentials, IP addresses, and configuration parameters are protected while still being accessible to the deployment workflows.

By setting up environment-specific secrets, the DEV, TEST, PRE-PROD, and PROD servers can be provisioned consistently without exposing critical data directly in the pipeline.

- **DEV Environment**
 - TOMCAT_HOST
 - TOMCAT_USER
 - TOMCAT_PASSWORD

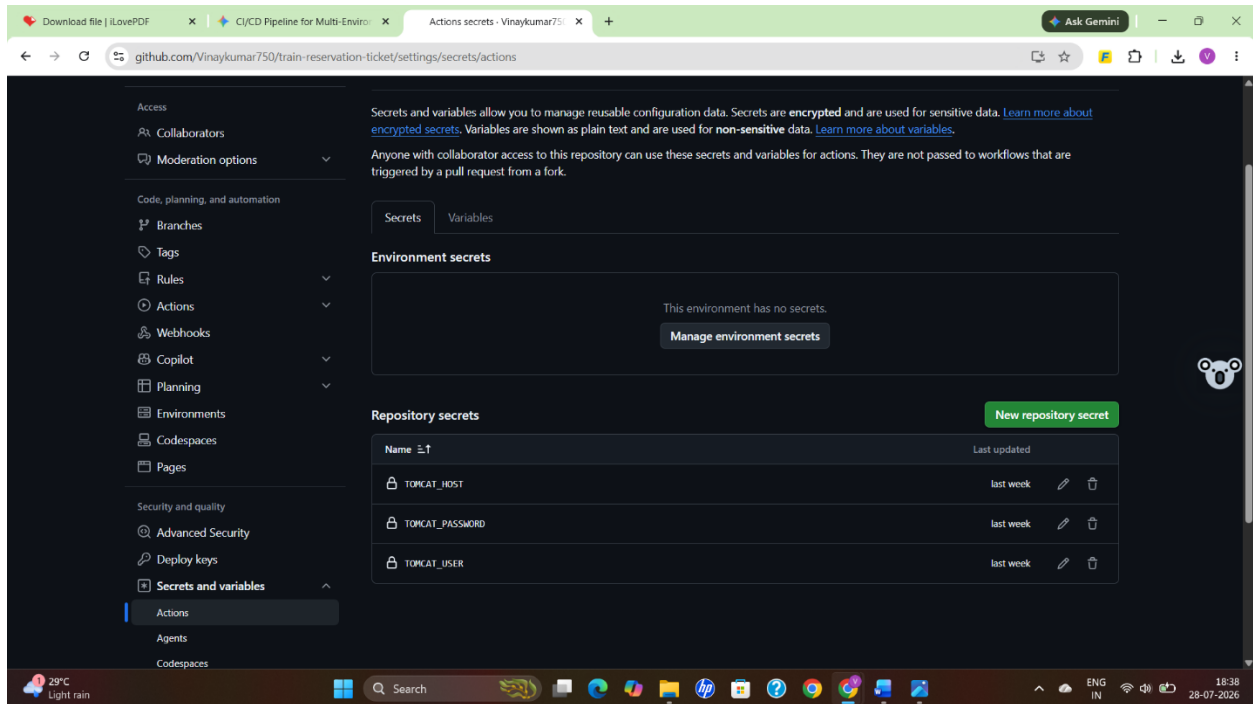


Fig: 5 Secrets & Variables

After you finish configuring the environment and secret variables in GitHub Actions, the next step is to update the workflow definition file located at `.github/workflows/tomcat.yaml` with the appropriate references to those secrets and environment settings.

Once the file has been modified to reflect the deployment requirements, commit the changes to the repository, which will automatically trigger the pipeline execution and initiate the deployment process.

GitHub Actions Pipeline Script

name: Build and Deploy to Tomcat

on:

push:

branches:

- main

jobs:

build-deploy:

runs-on: ubuntu-latest

steps:

- name: Checkout Source Code

uses: actions/checkout@v4

- name: Setup JDK 17

uses: actions/setup-java@v4

with:

distribution: 'temurin'

java-version: '17'

- name: Build WAR

run: mvn clean package

- name: Verify WAR File

run: ls -lh target/

- name: Deploy to All Tomcat Instances One by One

env:

TOMCAT_USER: \${ secrets.TOMCAT_USER }

TOMCAT_PASSWORD: \${ secrets.TOMCAT_PASSWORD }

Space-separated IP addresses of your 3 EC2 instances

TOMCAT_HOSTS: \${ secrets.TOMCAT_HOSTS }

run: |

Locate the .war file generated inside the target directory

WAR_FILE=\$(find target -name "*.war" | head -n 1)

echo "WAR file found: \$WAR_FILE"

Loop through each IP in the TOMCAT_HOSTS secret

for HOST in \$TOMCAT_HOSTS; do

echo "=====

echo "Deploying to Tomcat Instance: \$HOST"

echo "=====

curl -v \

-u "\$TOMCAT_USER:\$TOMCAT_PASSWORD" \

--upload-file "\$WAR_FILE" \

"http://\$HOST:8080/manager/text/deploy?path=/TrainBook&update=true"

echo "Waiting 5 seconds before deploying to the next instance..."

sleep 5

done

After the code was committed, the CI/CD pipeline ran successfully, and the deployments to the DEV, TEST, and PRE-PROD environments were automatically triggered and completed without any manual intervention.

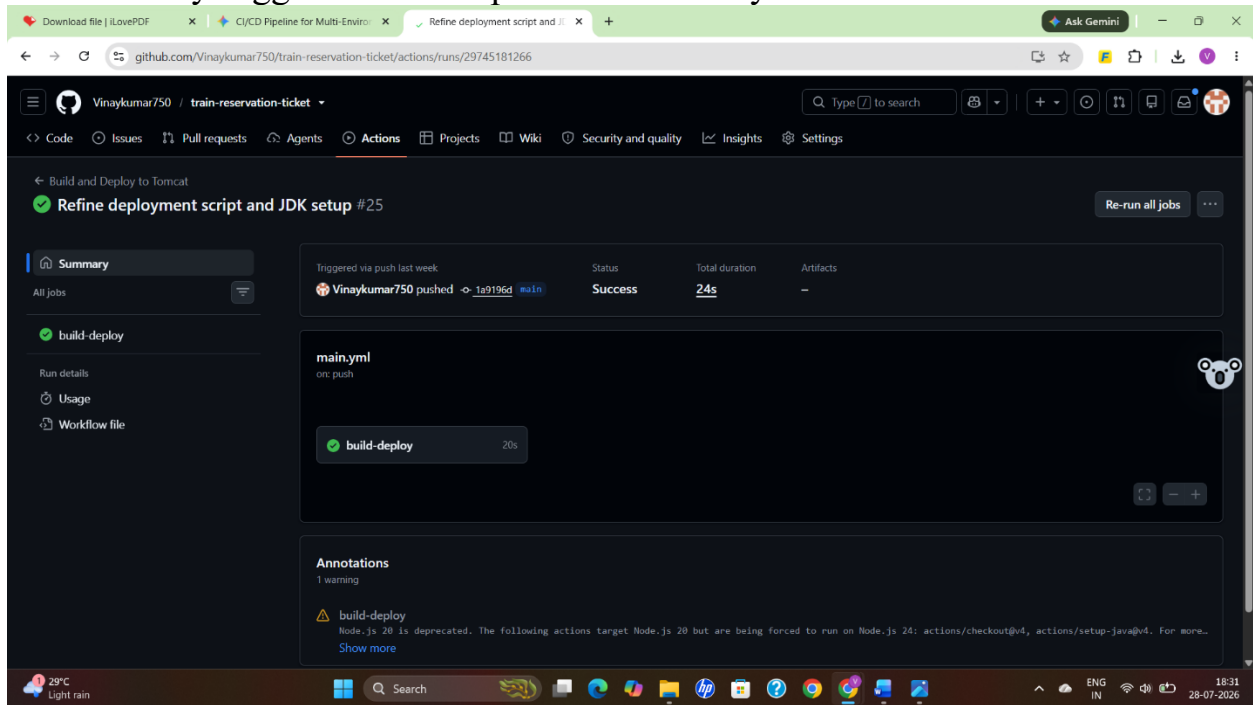


Fig: Deployment Pipeline

The PROD deployment, by design, was set to pause and await manual approval before continuing, thereby enforcing a controlled and secure release management process.

This safeguard ensures that production changes are reviewed and authorized, reducing risk and maintaining stability in the live environment

Application Deployed on all the Servers after Approval

The Java web application was deployed successfully across all four environments—DEV, TEST, PRE-PROD, and PROD—ensuring uniform functionality and consistent configuration on each Apache Tomcat server hosted within AWS EC2

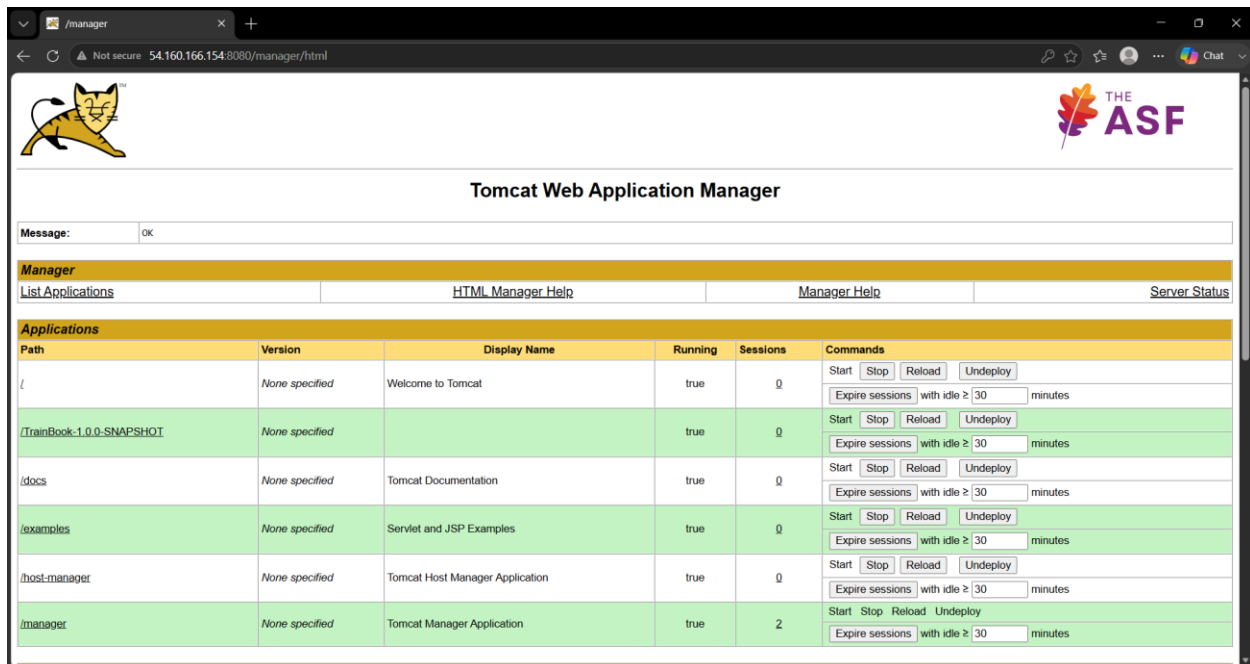
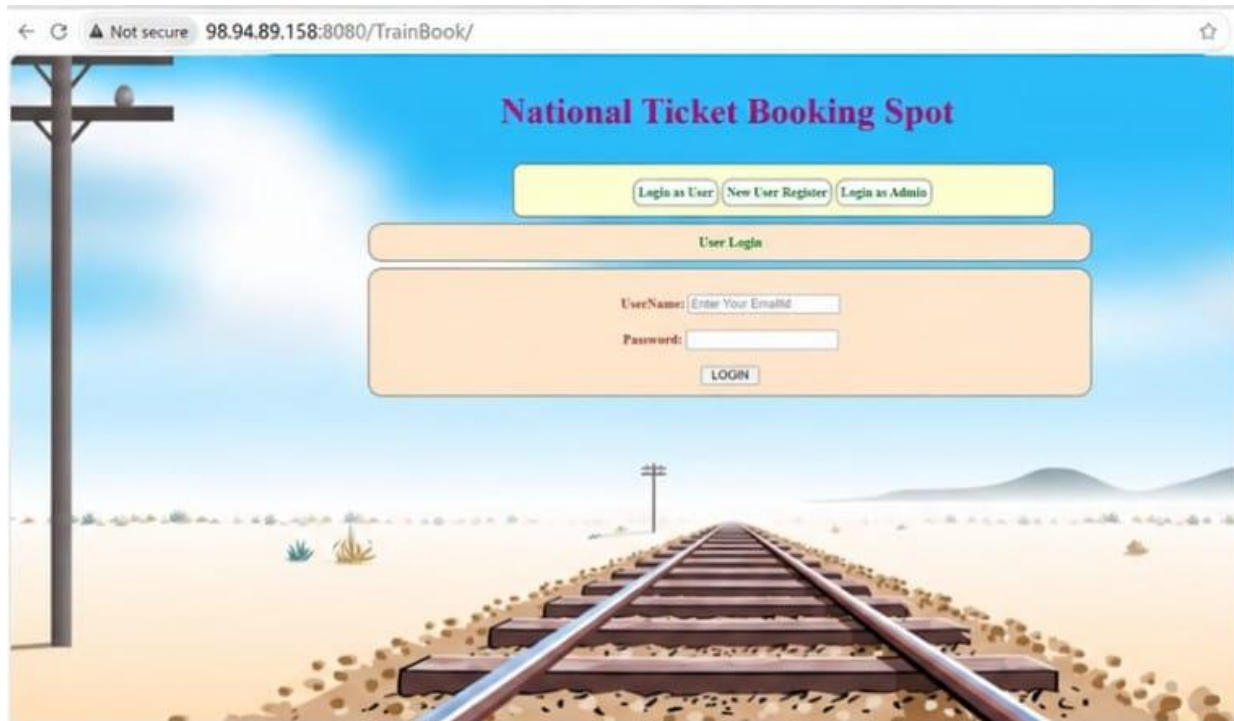


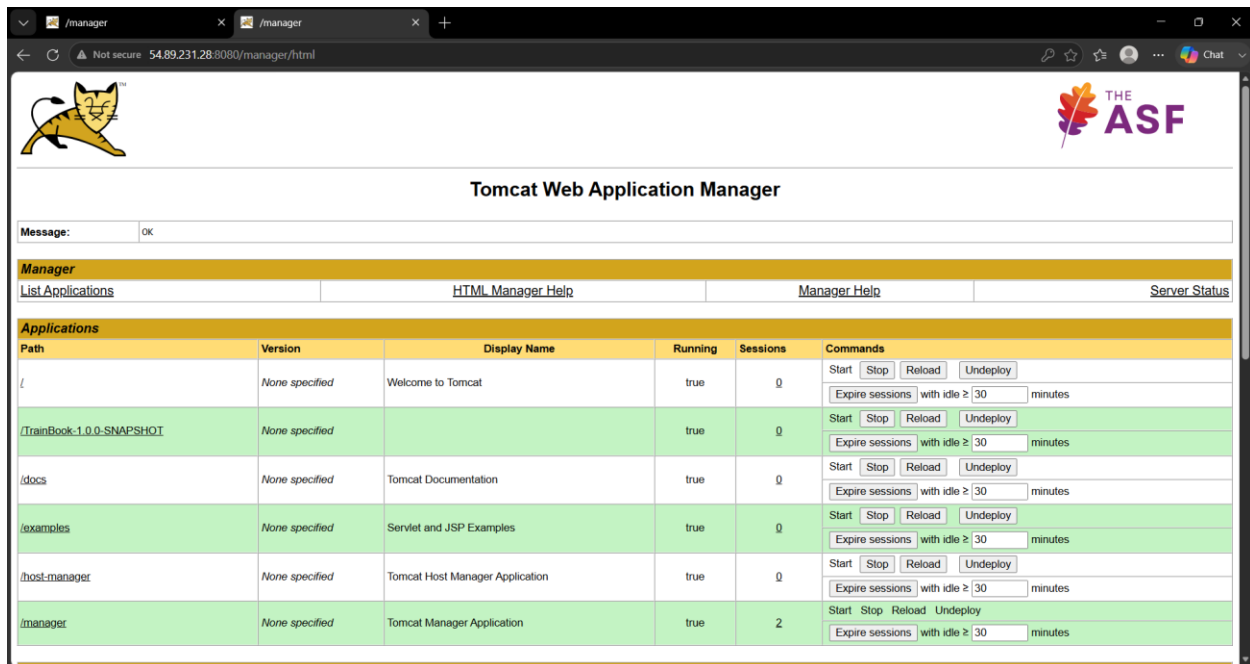
Fig: 7 Application Deployed into Tomcat

Deployment Workflow Across Environments

Deployment to the **DEV environment** is configured to run automatically without requiring any manual approval.



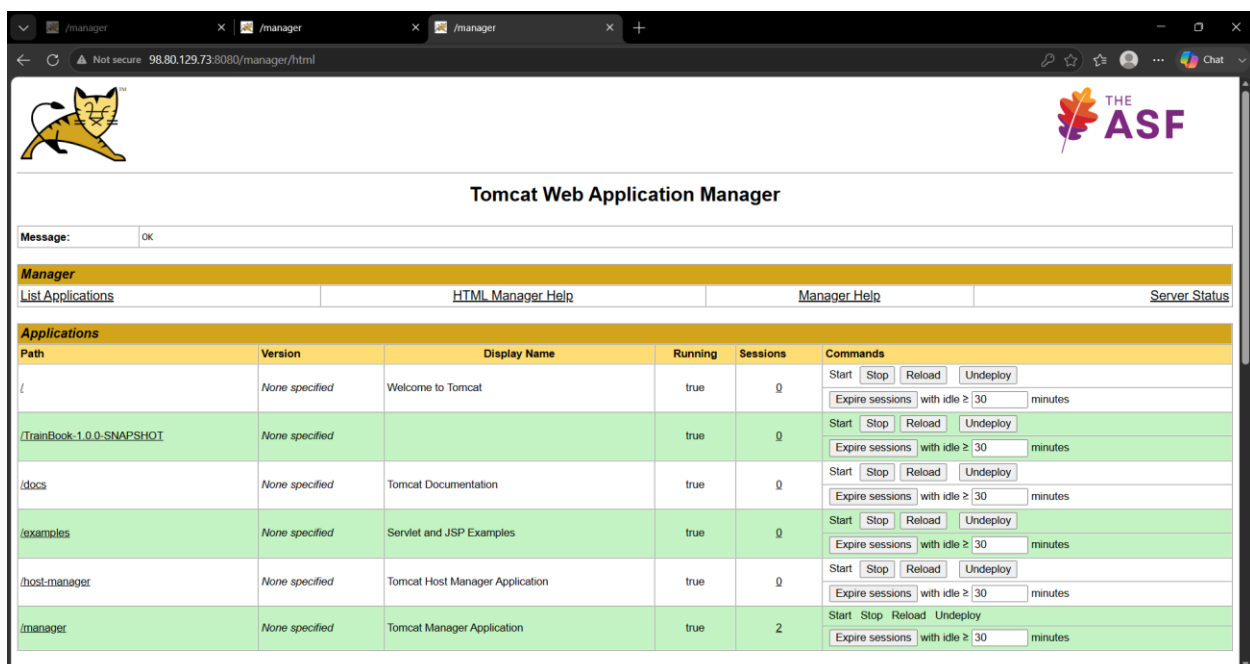
TEST Environment-Deployment to the **TEST** environment runs automatically without requiring manual approval, enabling quick validation of new changes.



The screenshot shows the Tomcat Web Application Manager interface. At the top, there is a message bar and navigation links: [List Applications](#), [HTML Manager Help](#), [Manager Help](#), and [Server Status](#). Below this is a table titled "Applications" with the following columns: Path, Version, Display Name, Running, Sessions, and Commands.

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/TrainBook-1.0.0-SNAPSHOT	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	2	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

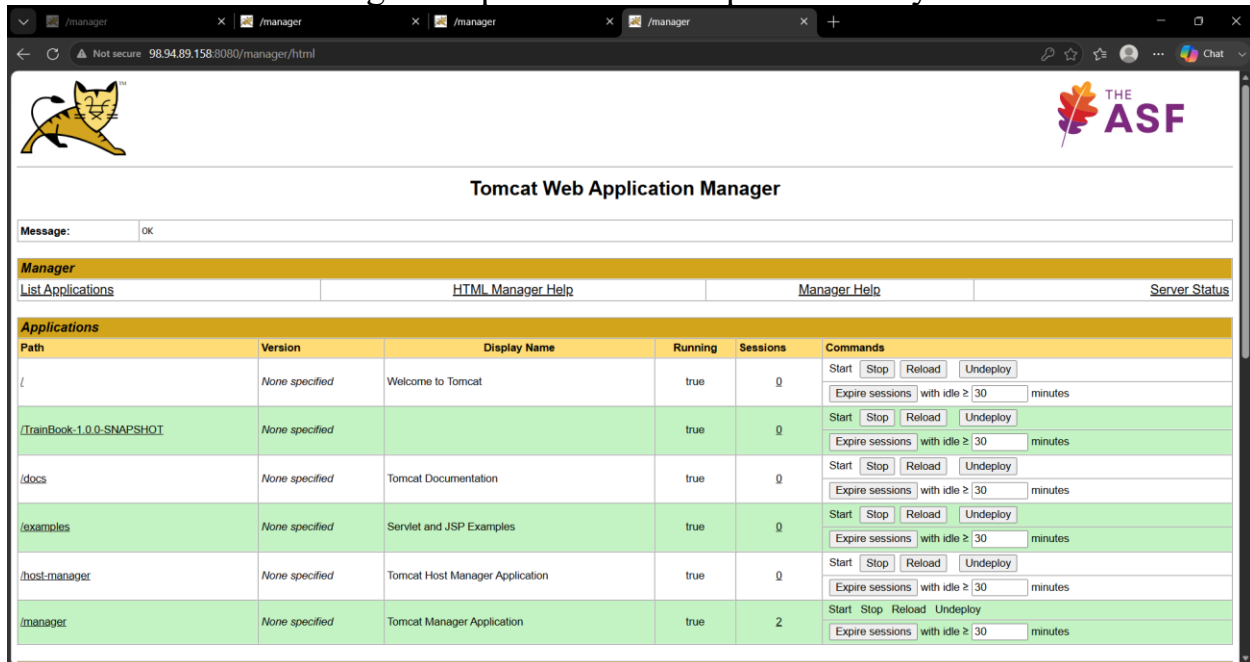
PRE-PROD Environment-Deployment to the **PRE-PROD** environment also runs automatically without requiring manual approval, allowing teams to verify stability before production release.



The screenshot shows the Tomcat Web Application Manager interface, identical to the one above. It displays the same table of applications with their paths, versions, display names, running status, session counts, and available commands.

Path	Version	Display Name	Running	Sessions	Commands
/	None specified	Welcome to Tomcat	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/TrainBook-1.0.0-SNAPSHOT	None specified		true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/docs	None specified	Tomcat Documentation	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/examples	None specified	Servlet and JSP Examples	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/host-manager	None specified	Tomcat Host Manager Application	true	0	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes
/manager	None specified	Tomcat Manager Application	true	2	Start Stop Reload Undeploy Expire sessions with idle ≥ 30 minutes

PROD Environment-Deployment to the **PROD environment** is gated by a **manual approval process**. This safeguard ensures that only reviewed and validated code changes are promoted to the production system.



Project Conclusion

When a developer commits or pushes code to GitHub, the **GitHub Actions workflow** defined in **tomcat.yaml** is automatically triggered. The developer then selects the target environment—DEV, TEST, PRE-PROD, or PROD.

The **CI pipeline** compiles the application with Maven and generates a .war file, while the **CD pipeline** securely transfers this artifact to the designated Apache Tomcat server hosted on AWS EC2.

Tomcat subsequently deploys the application from the `webapps/` directory. For higher environments, such as PROD, **manual approval steps** are enforced to provide an additional layer of verification, ensuring controlled and secure deployments while preventing accidental releases.

--Thank You--
R. Vinay Kumar